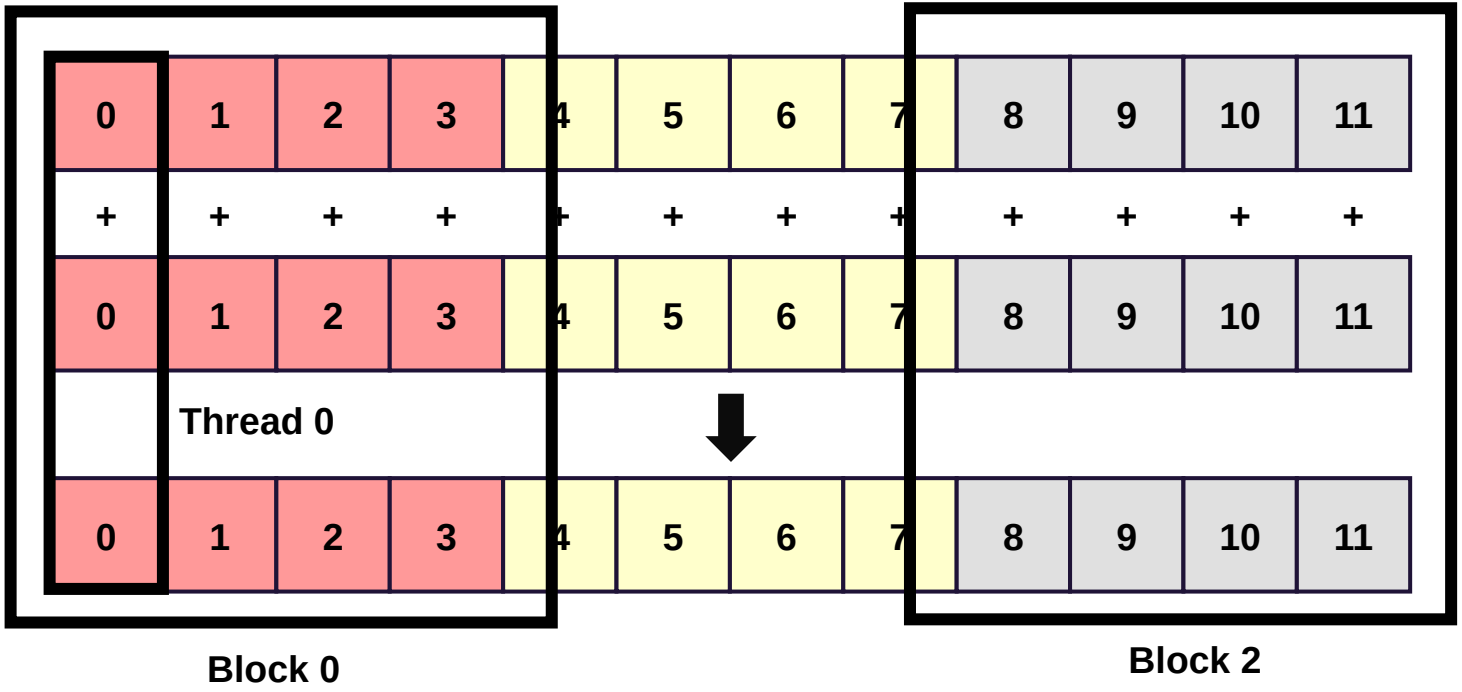
CSE 554 Machine Learning Systems

Lecture 3 – GPU Kernel Optimizations - 2

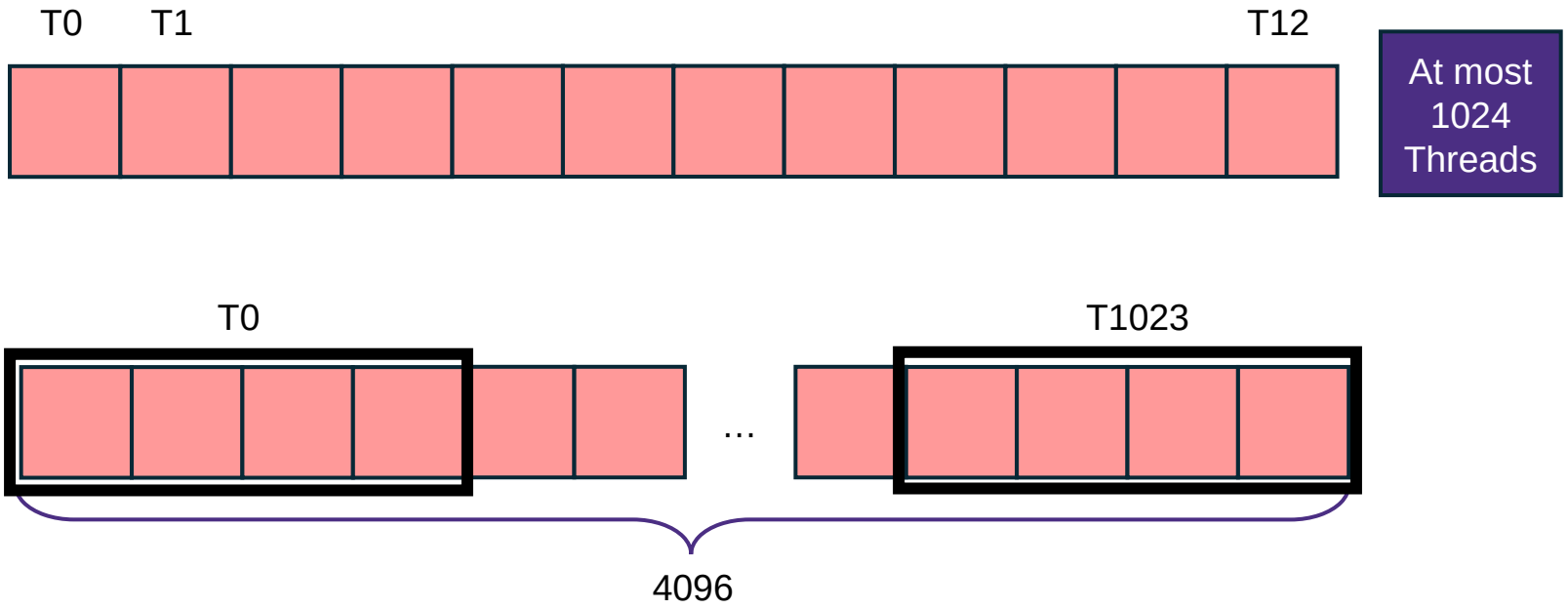
Baris Kasikci

BE BOUNDLESS

CUDA Addition Kernel - Thread Assignment



Handling Long Arrays



Class Overview

1. **GPU Architecture and Programming**
2. Transformers (Architecture and Implementation)
3. Performance modeling
4. Quantization and Sparsity
5. Memory Management
6. Mixture of Experts (MoE)
7. Retrieval-Augmented Generation (RAG)
8. Batching
9. Speculative decoding
10. Parallelism
11. End-to-end serving
12. Agentic systems
13. Large-scale deployment

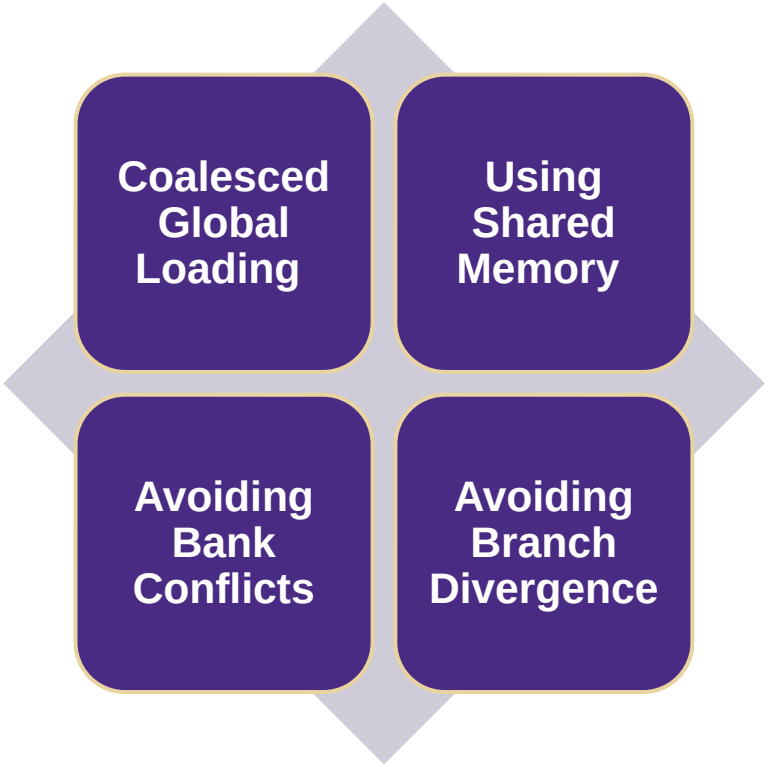
Additional CUDA features in modern GPUs

- Unified memory address (P100+)
- NvLink (P100+)
- Clusters (H100+)
  - Make inter-SM communication feasible
- TMA (H100+)
  - Asynchronous loads form global memory
- NVSHARP (H100+)
- New data types: FP4 and FP6 (B100+)

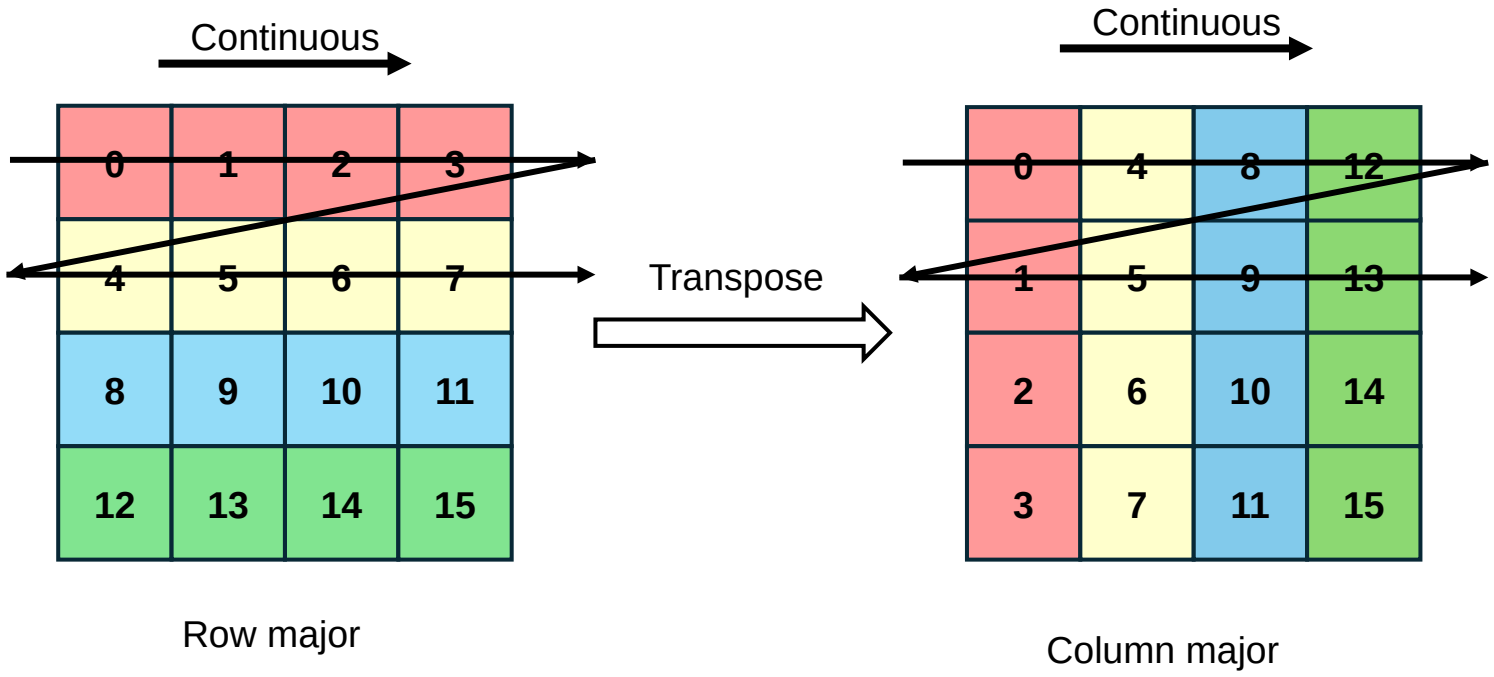
Today

- Learn the best practices of GPU programming
  - Coalesced loading
  - Using shared memory
  - Avoiding bank conflicts
  - Avoiding branch divergence
- Learn the state-of-the-art libraries
- Learn how to use Python bindings to expose kernels to Python

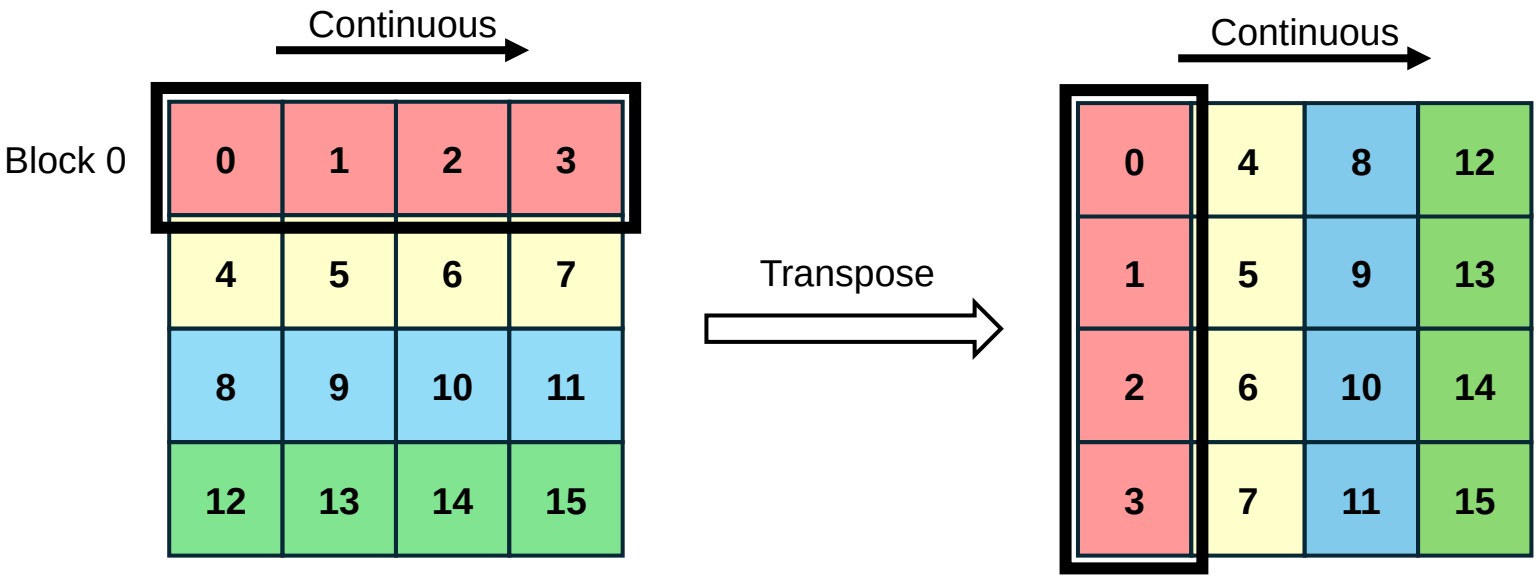
How to write fast kernels?



Matrix Transpose Kernel

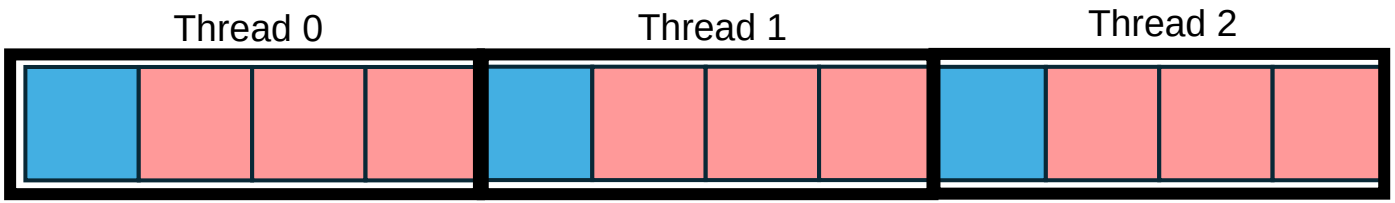


Transpose V1: Row-Wise Partitioning



https://developer.nvidia.com/blog/efficient-matrix-transpose-cuda-cc/

Transpose V1: ~3 ms



Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	1,146,880	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.10	Avg. Divergent Branches	0

Uncoalesced Global Accesses
Est. Speedup: 86.35%

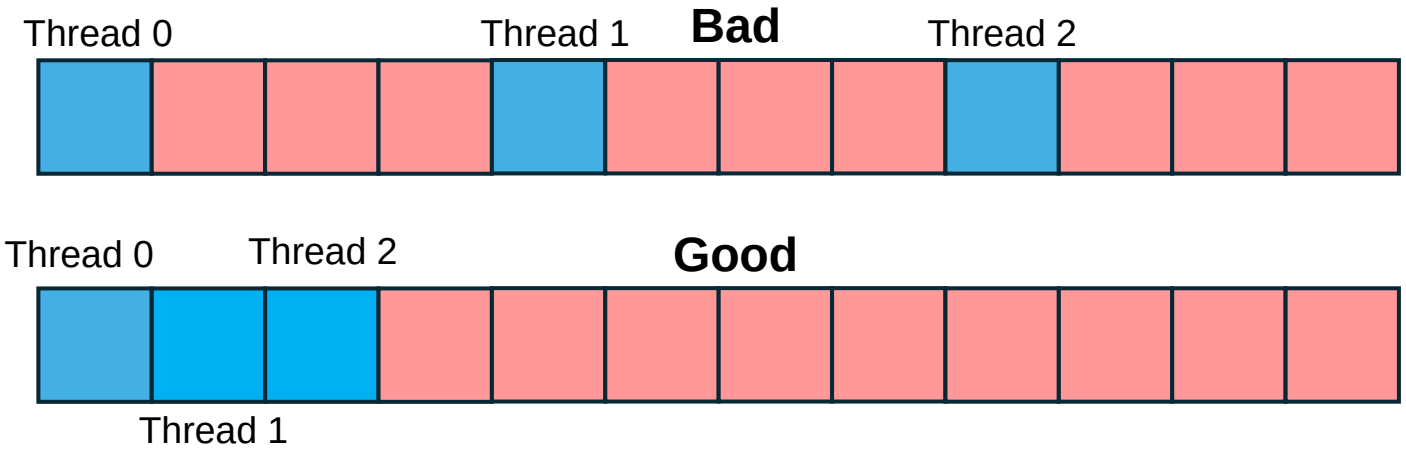
This kernel has uncoalesced global accesses resulting in a total of 117440512 excessive sectors (88% of the total 134217728 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7f00fb99f990 in transpose	14,680,064	13
0x7f00fb99f970 in transpose	14,680,064	13
0x7f00fb99f960 in transpose	14,680,064	13
0x7f00fb99f920 in transpose	14,680,064	13
0x7f00fb99f910 in transpose	14,680,064	13

Transpose V2: Global memory coalescing

- Inside one warp, if the memory accesses are contiguous, the memory access is coalesced (batched)
- Data can be retrieved from global memory in one or a few transactions



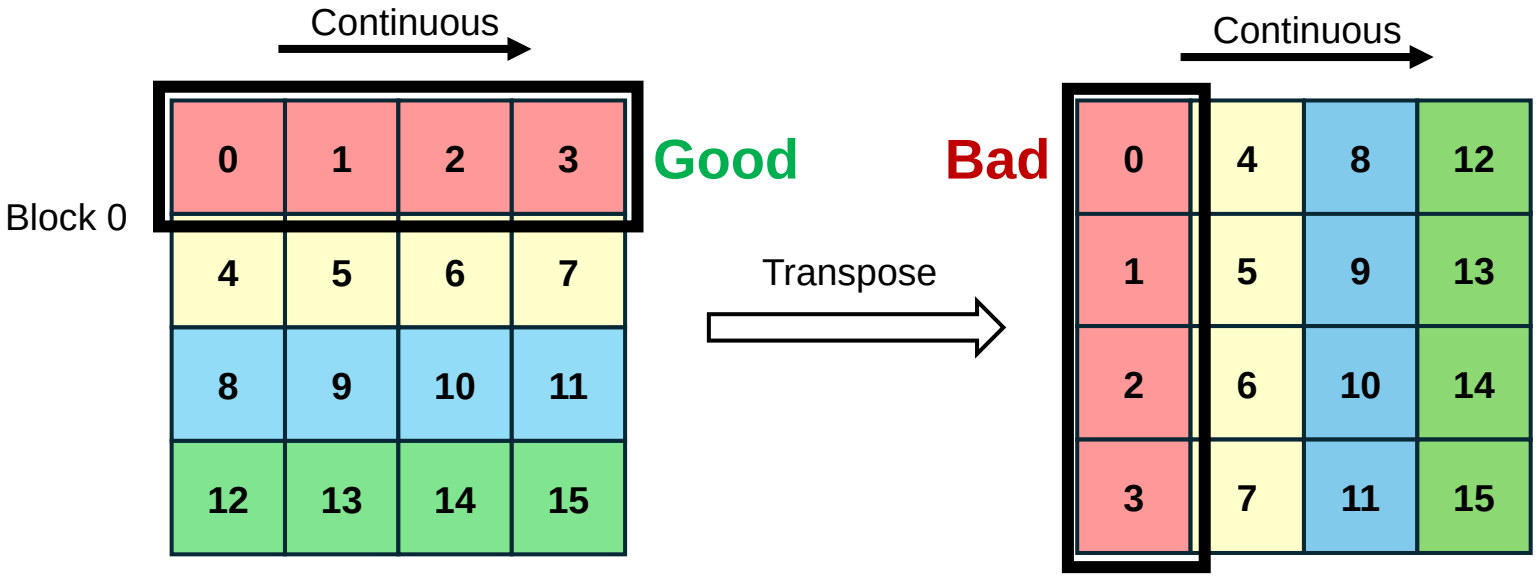
Transpose V2:1.40 ms: Uncoalesced Writes to the Output Matrix

Uncoalesced Global Accesses
Est. Speedup: 77.52%

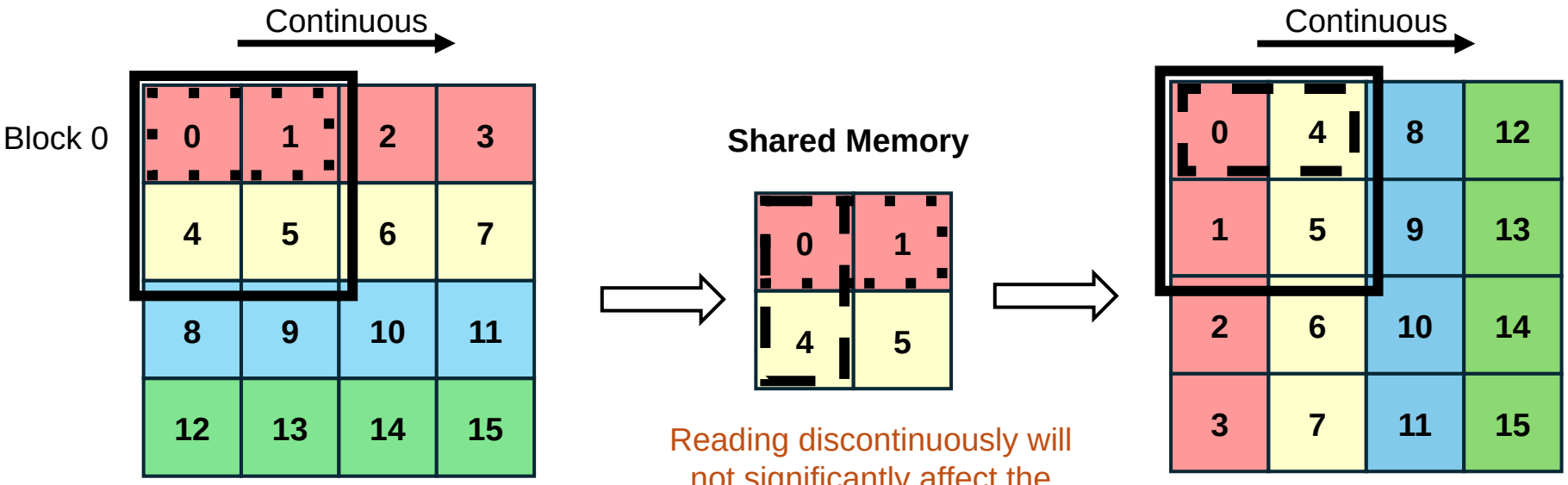
This kernel has uncoalesced global accesses resulting in a total of 58720256 excessive sectors (78% of the total 75497472 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7f6fc799f720 in transpose	58,720,256	100
0x7f6fc799f6d0 in transpose	0	0



Transpose V3: Tilewise partitioning



Reading discontinuously will not significantly affect the shared memory performance (shared memory is fast)

Shared Memory Allocation

- Static allocation
  - Inside the kernel, call
    - `__shared__ float f_array[10];`
  - Easier to use
  - Fixed size, up to 48 KB
- Dynamic allocation
  - Inside the kernel call
    - `extern __shared__ int shared_mem[];`
      - No size specified yet; this will be done on the host
  - When launching the kernel, call
    - `my_kernel<<<grid, block, shared_mem_size_in_bytes>>>`
  - Above 48KB and up to 228 KB
    - `cudaFuncSetAttribute(my_kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, desiredSizeInBytes);`

Transpose V3 312 us: Bank Conflict

Memory Workload Analysis — Memory Chart

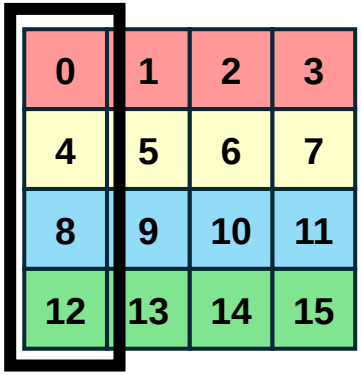
Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

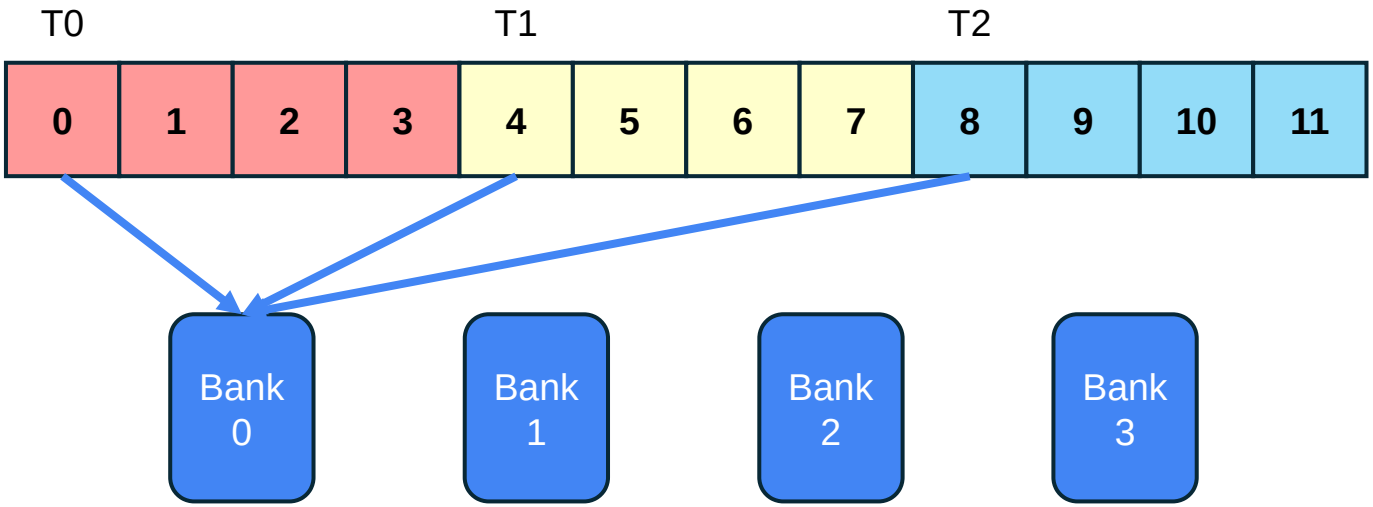
Memory Throughput [Tbyte/s]	2.13	Mem Busy [%]	48.15
L1/TEX Hit Rate [%]	0	Max Bandwidth [%]	63.53
L2 Hit Rate [%]	50.95	Mem Pipes Busy [%]	13.83
L2 Compression Input Sectors	8,389,199	L2 Compression Success Rate [%]	0
L2 Compression Ratio [%]	0	-	-

Low Compression Rate
Est. Speedup: 22.20%
Out of the 268454368.0 bytes sent to the L2 Compression unit only 0.00% were successfully compressed. To increase this success rate, consider marking only those memory regions as compressible that contain the most zero values and/or expose the most homogeneous values.

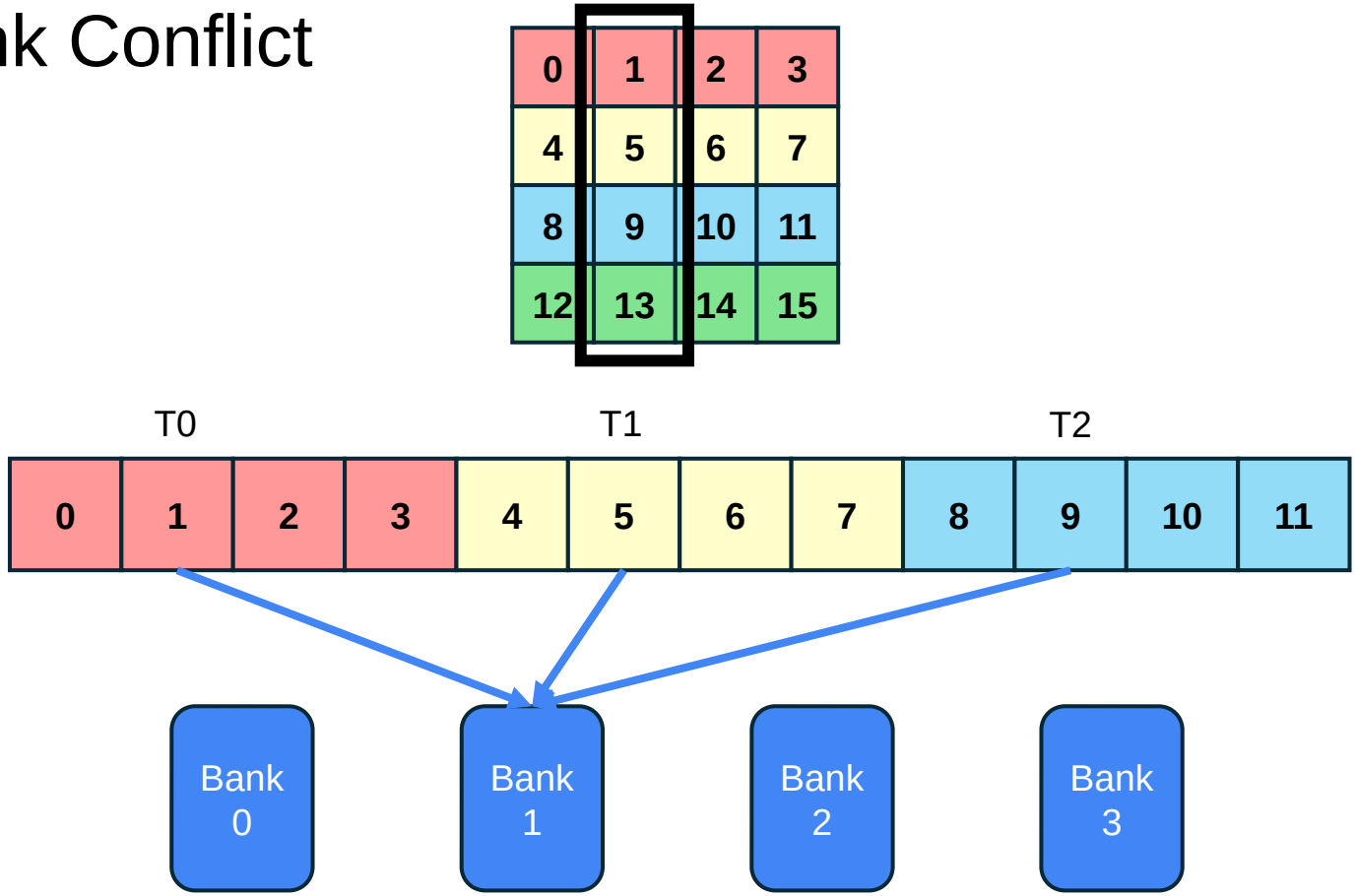
Shared Load Bank Conflicts
Est. Speedup: 43.79%
The memory access pattern for shared loads might not be optimal and causes on average a 33.0 - way bank conflict across all 524288 shared load requests.This results in 15197160 bank conflicts, which represent 87.87% of the overall 17294312 wavefronts for shared loads. Check the Source Counters section for uncoalesced shared loads.

Bank Conflict



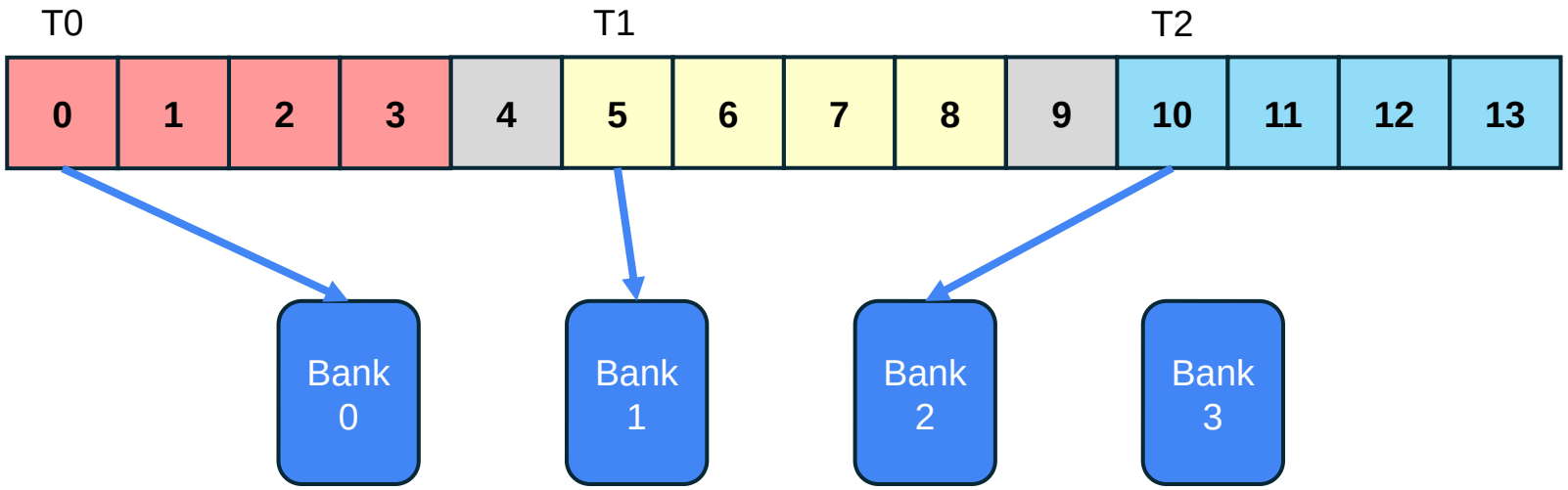


Bank Conflict

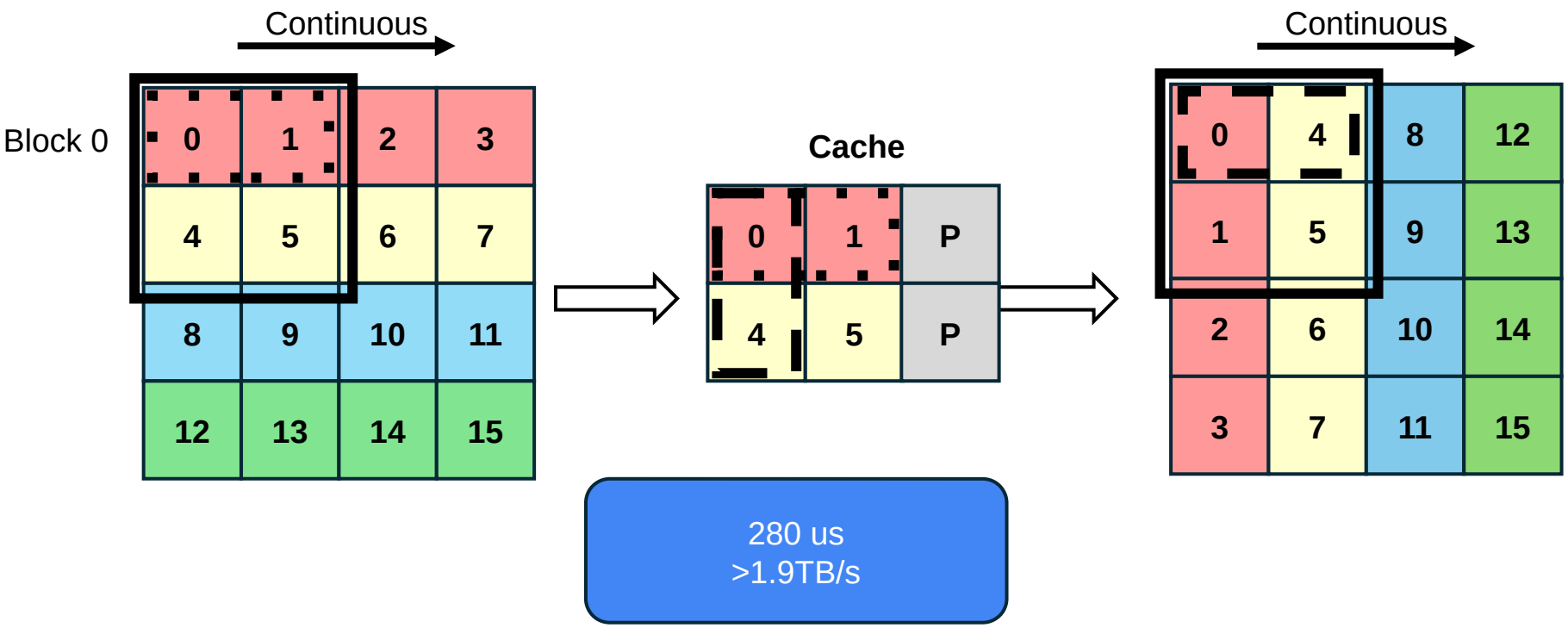


Bank Conflict

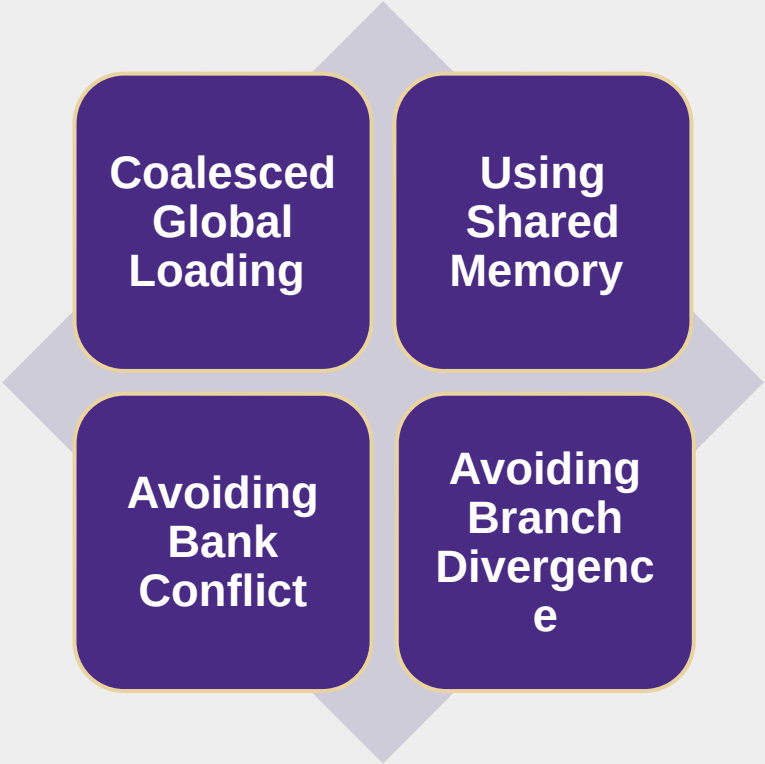
0	1	2	3	P
4	5	6	7	P
8	9	10	11	P
12	13	14	15	P



Transpose V4: Padding



More complex problem: Reduction



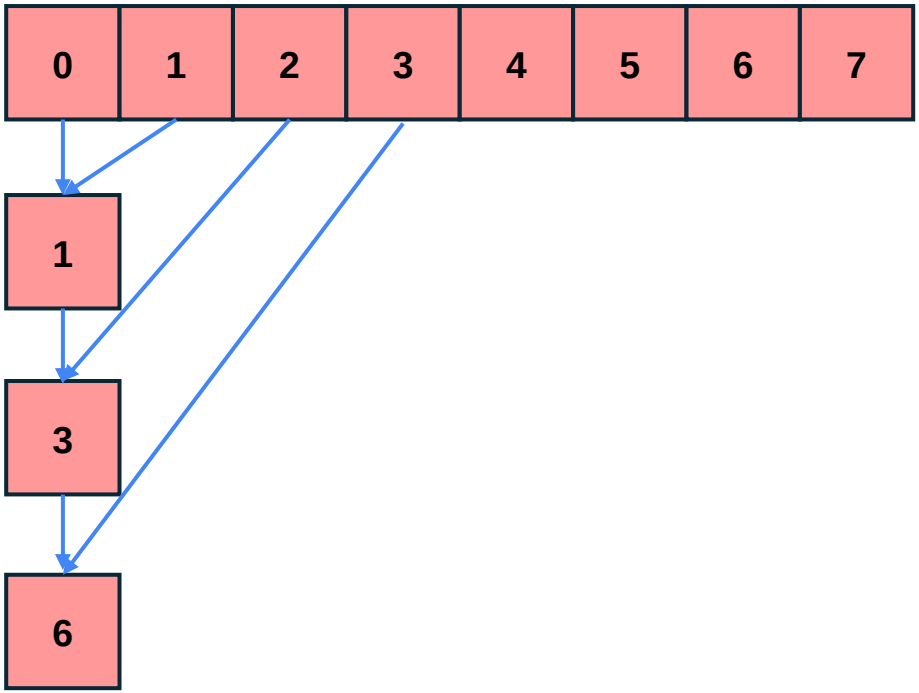
Reduction Problem

- Reduction example:

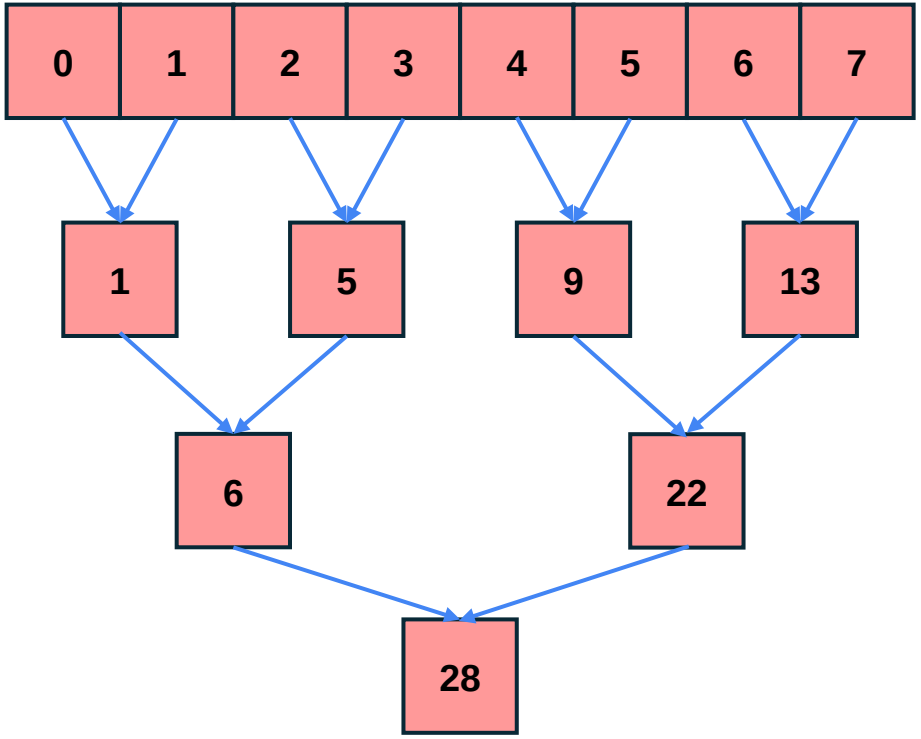
  for elements in array:

  Temp = op (temp, element)

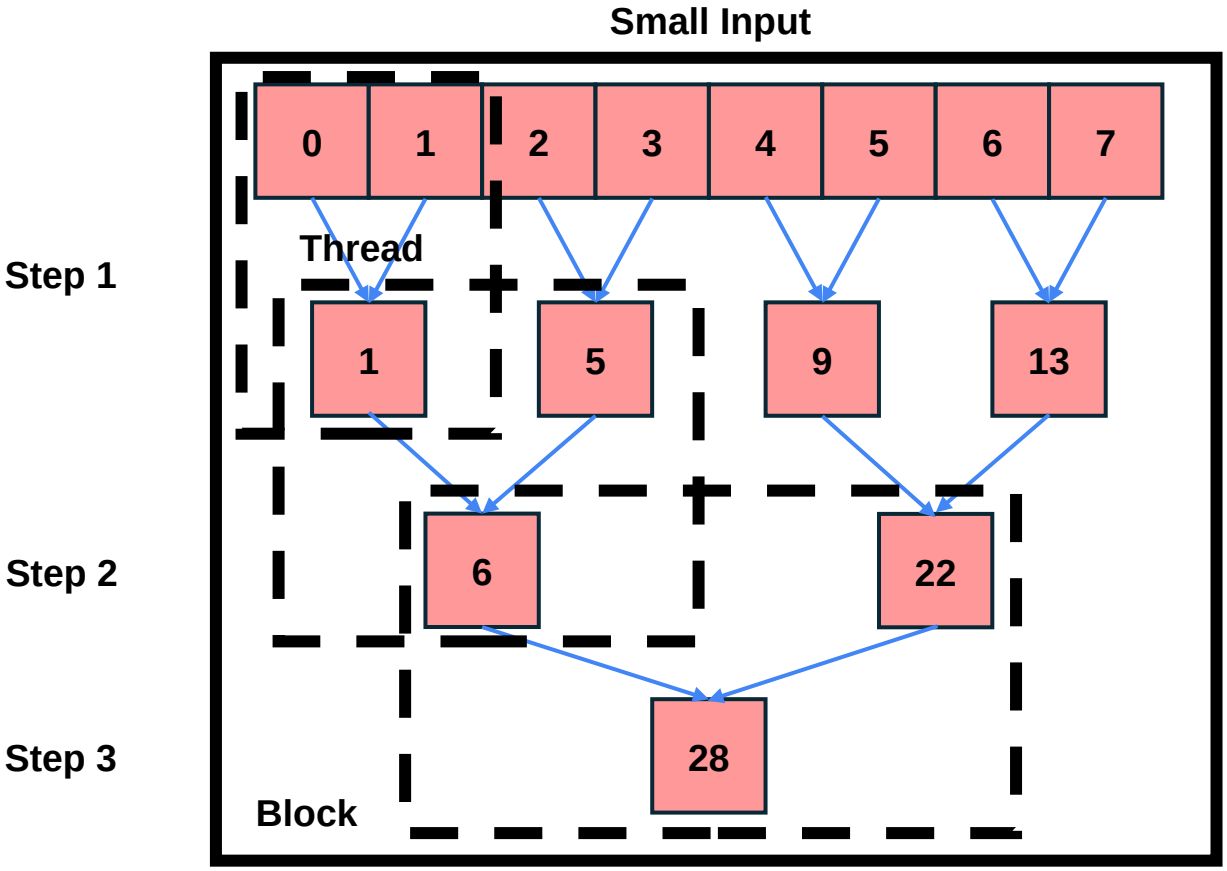
- Examples of reduction
  - Sum
  - Max / Min
  - …



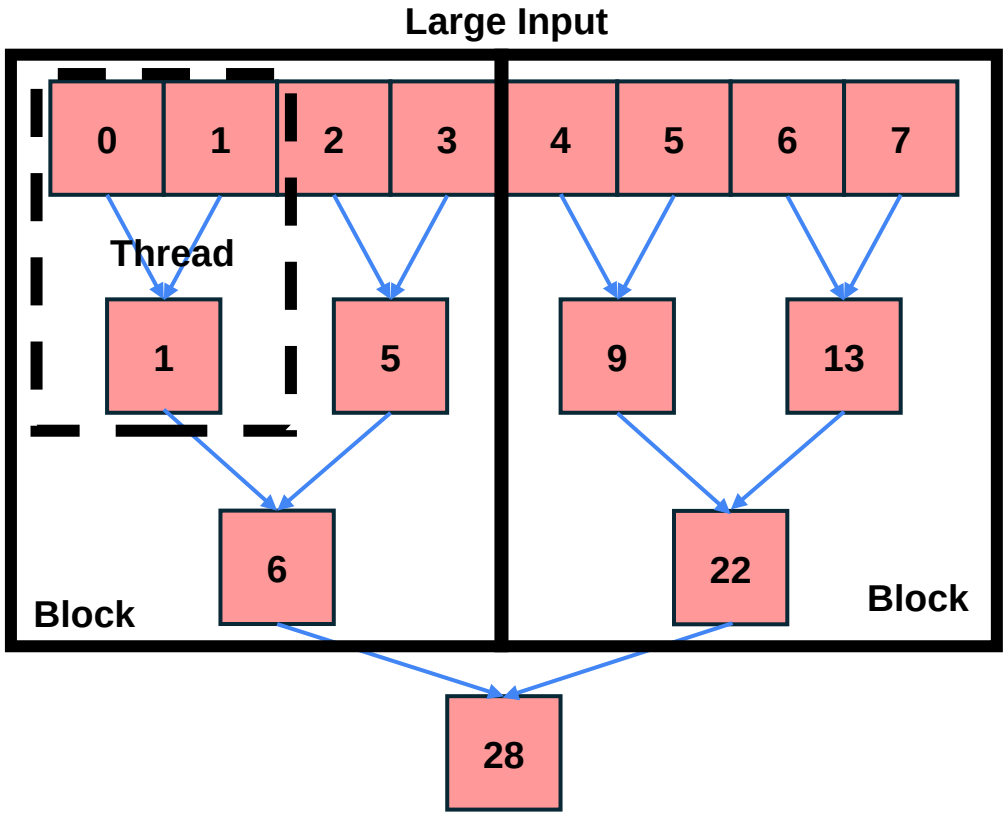
Parallel Reduction



Global View of the Reduction Kernel



Global View of the Reduction Kernel



Reduction #1

- Step i
  - threadID % 2^N == 0 does the work
  - stride = 2 ^ (N-1)

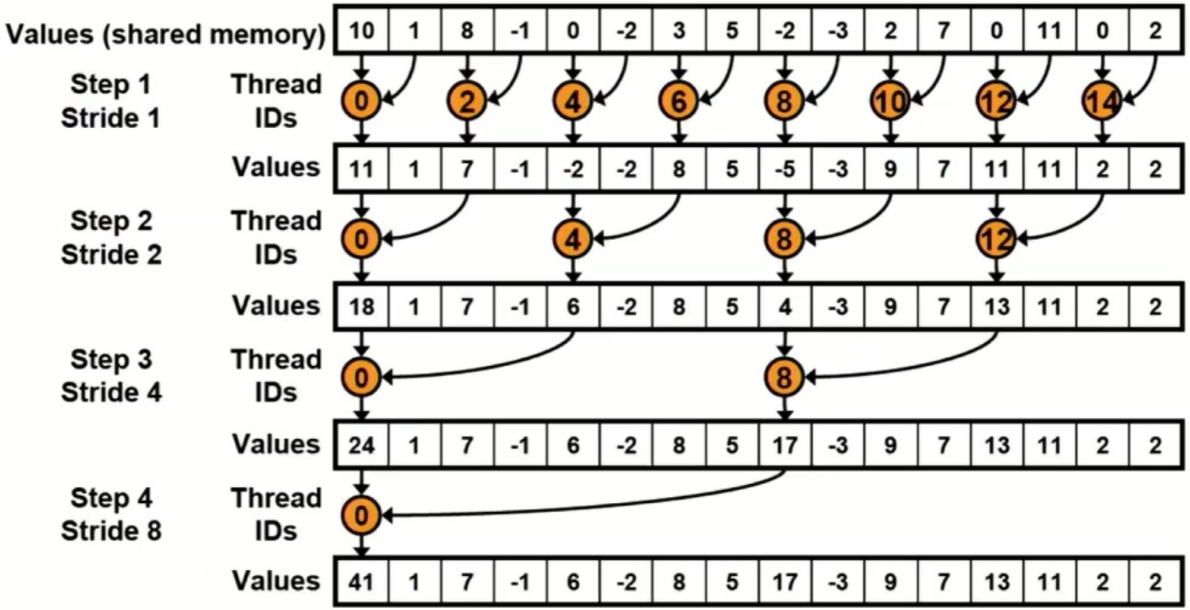
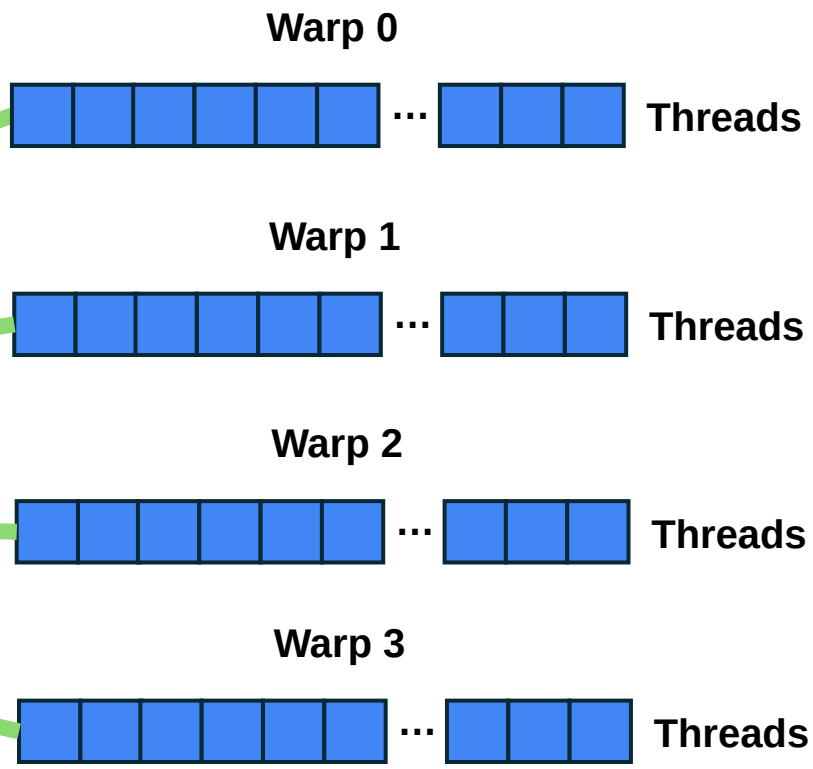


Figure taken from Caltech CS 179: GPU Programming Lecture 7

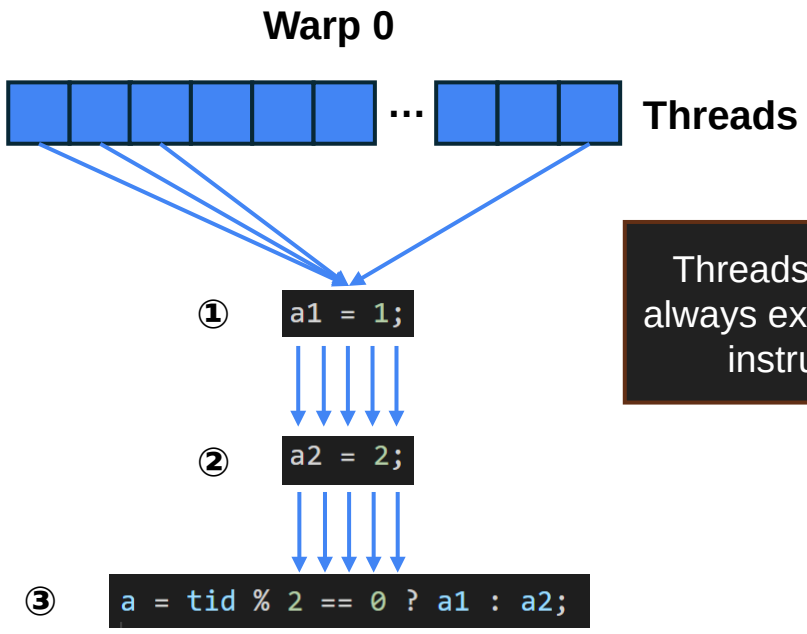
Handling Branches in CUDA

```cuda
__global__ void kernel() {
    int tid = threadIdx.x;
    int a;
    if (tid < 32) {
        a = 1;
    } else if (tid < 64) {
        a = 2;
    } else if (tid < 96) {
        a = 3;
    } else {
        a = 4;
    }
}
```



Handling Branches in CUDA

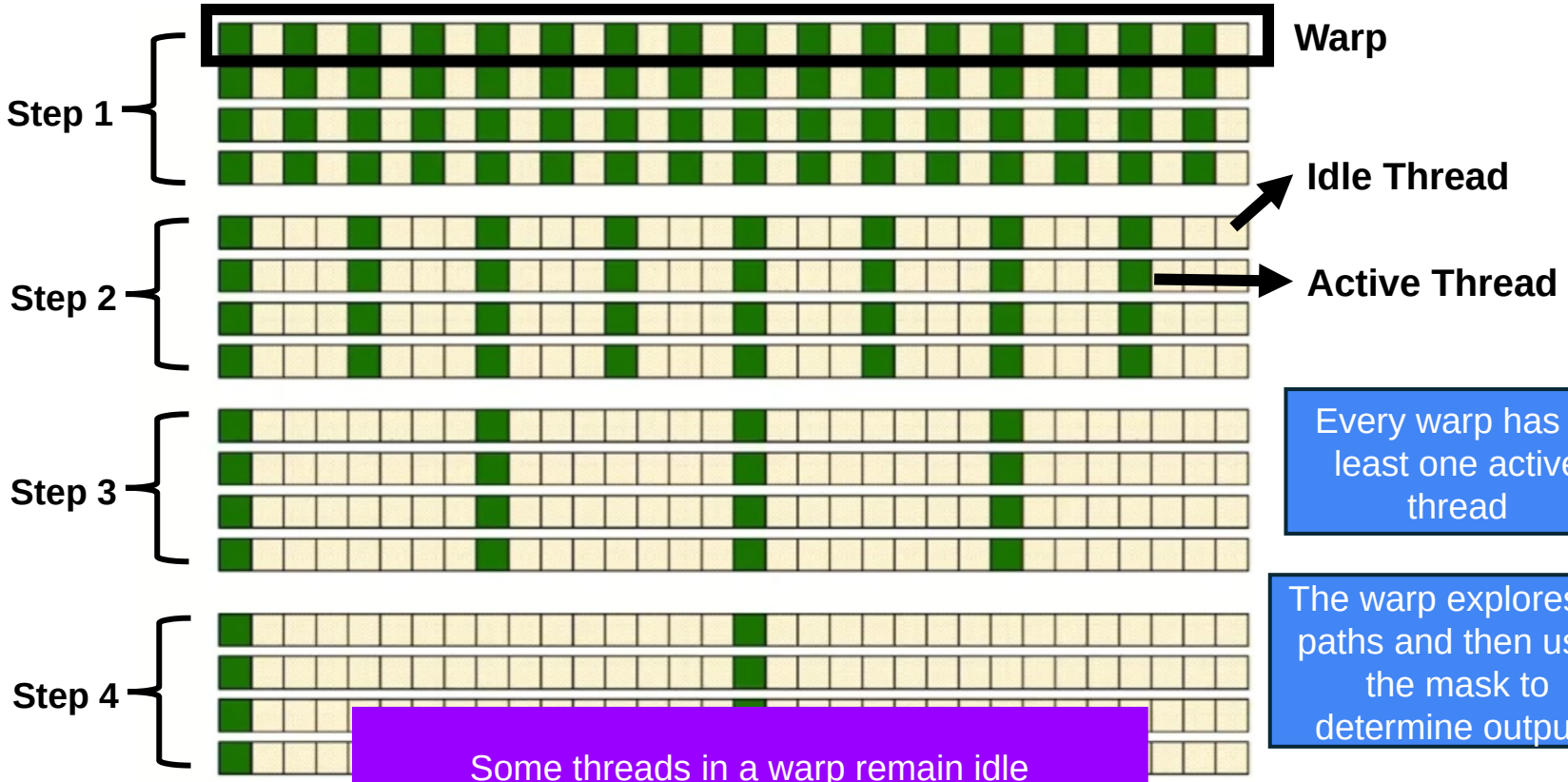
```cpp
__global__ void kernel2() {
    int tid = threadIdx.x;
    int a;
    if (tid % 2 == 0) {
        a = 1;
    } else {
        a = 2;
    }
}
```



Threads in a warp always execute same instructions

Explore all paths and then use mask to determine outputs

Branch Divergence

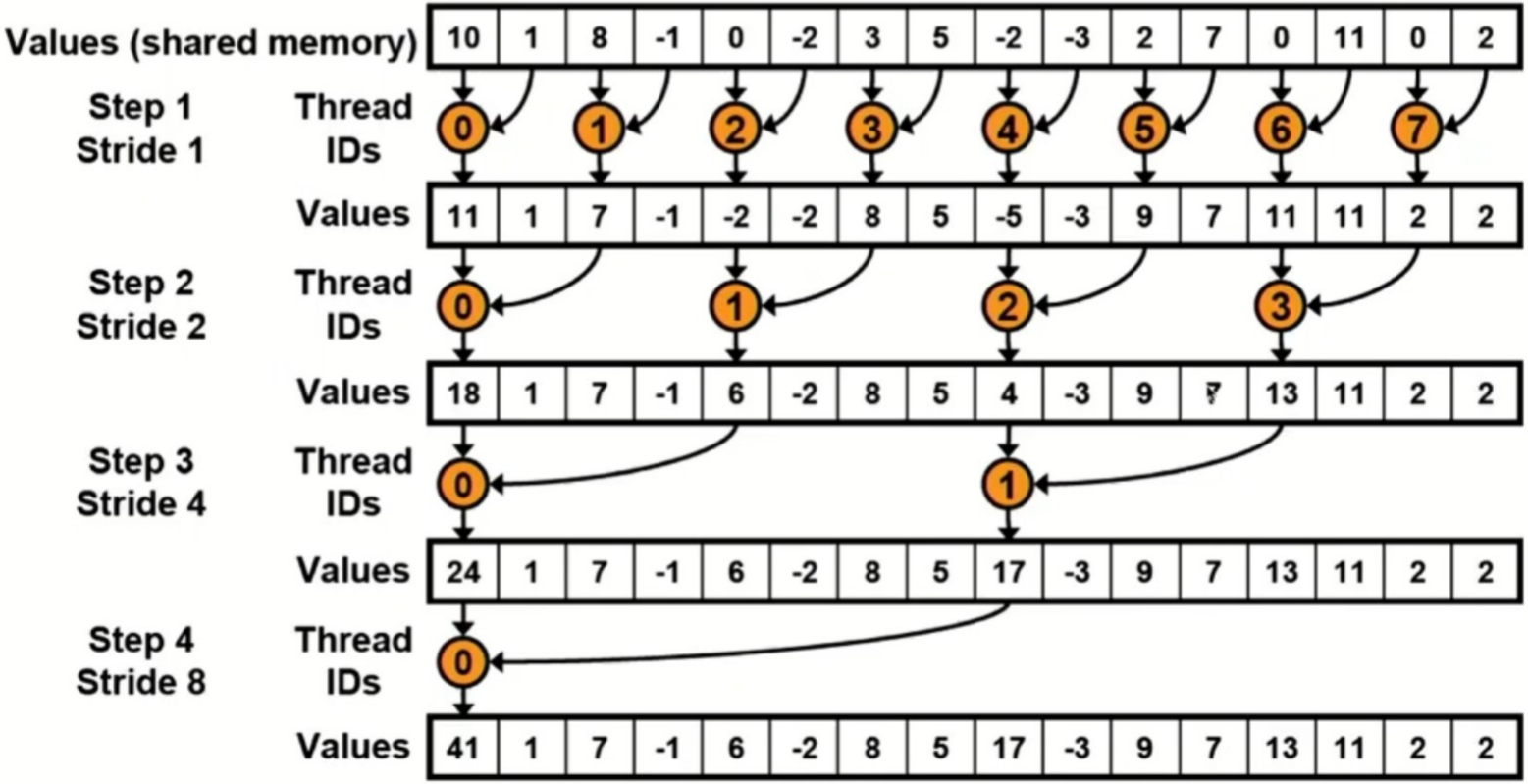


Every warp has at least one active thread

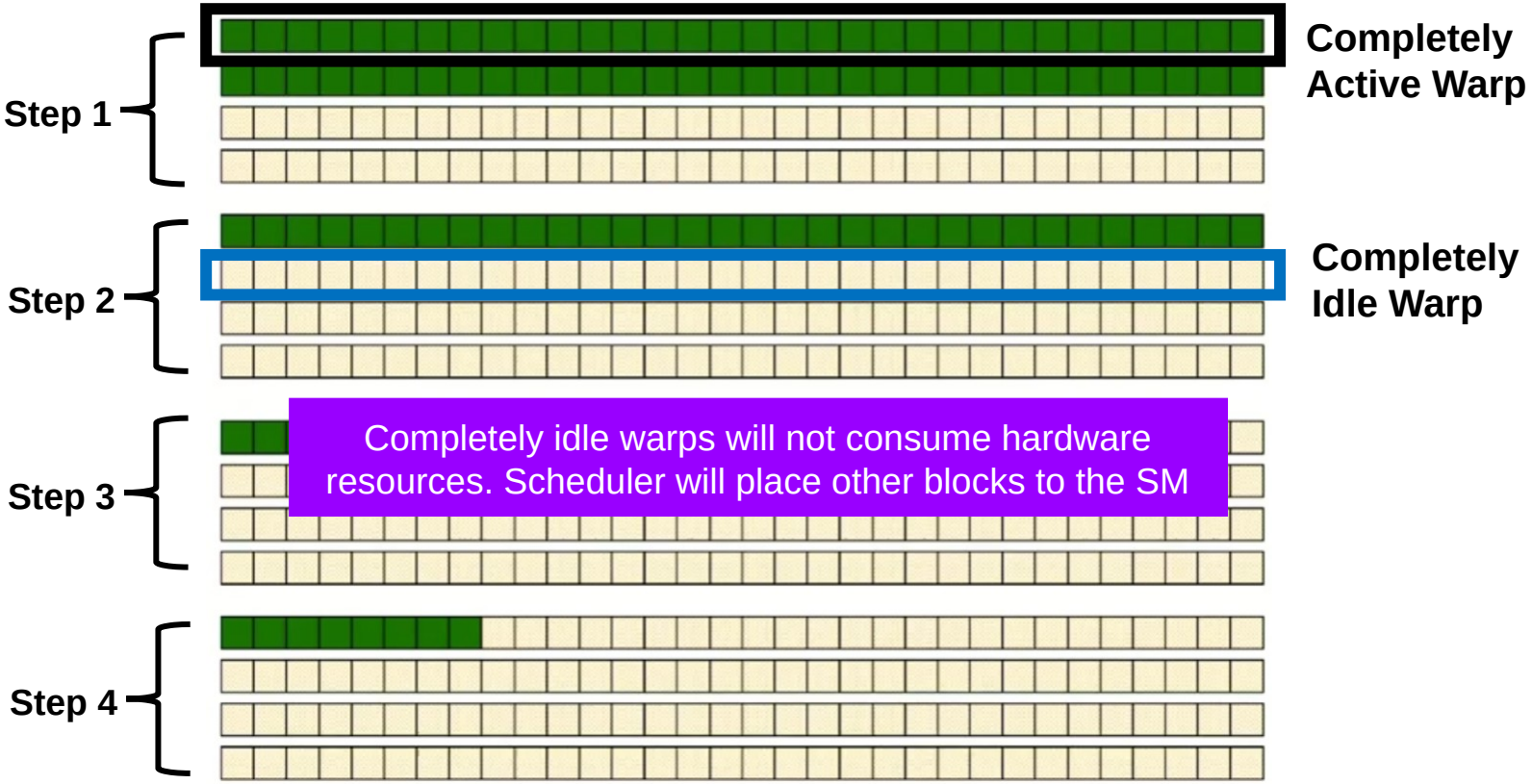
The warp explores all paths and then uses the mask to determine outputs

Some threads in a warp remain idle

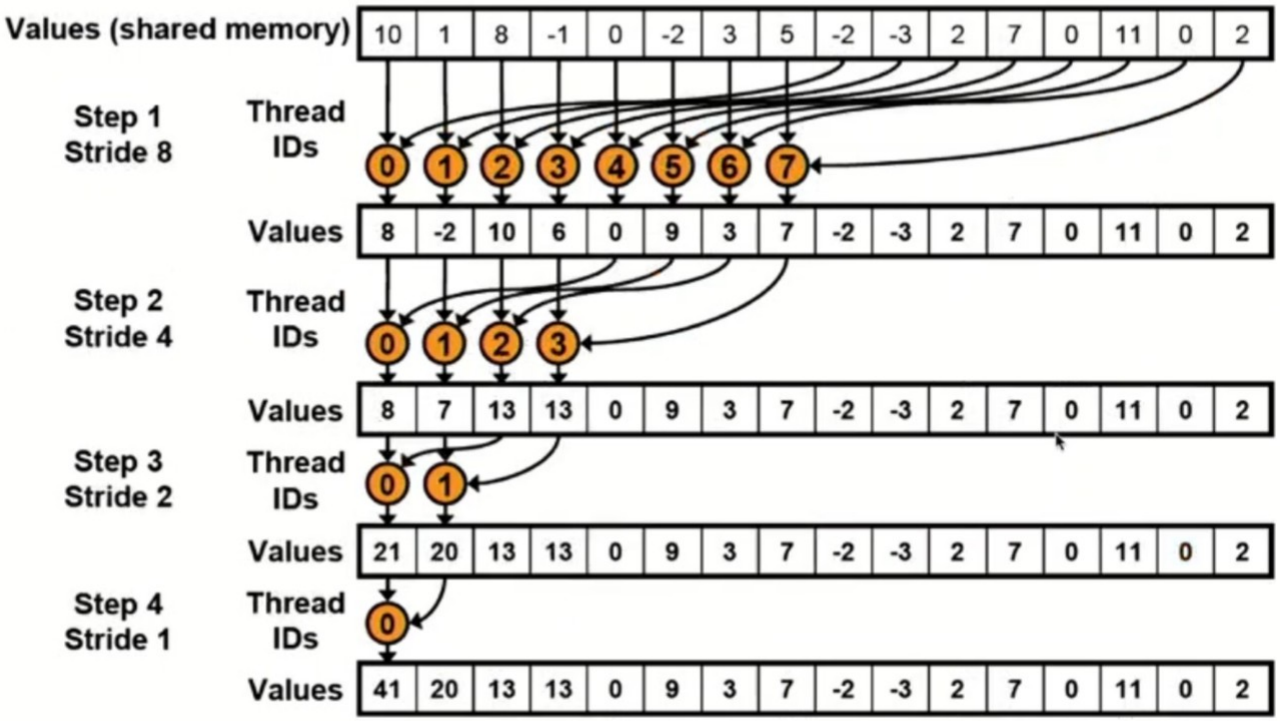
Reduction #2: Access Pattern



Resolved Divergence

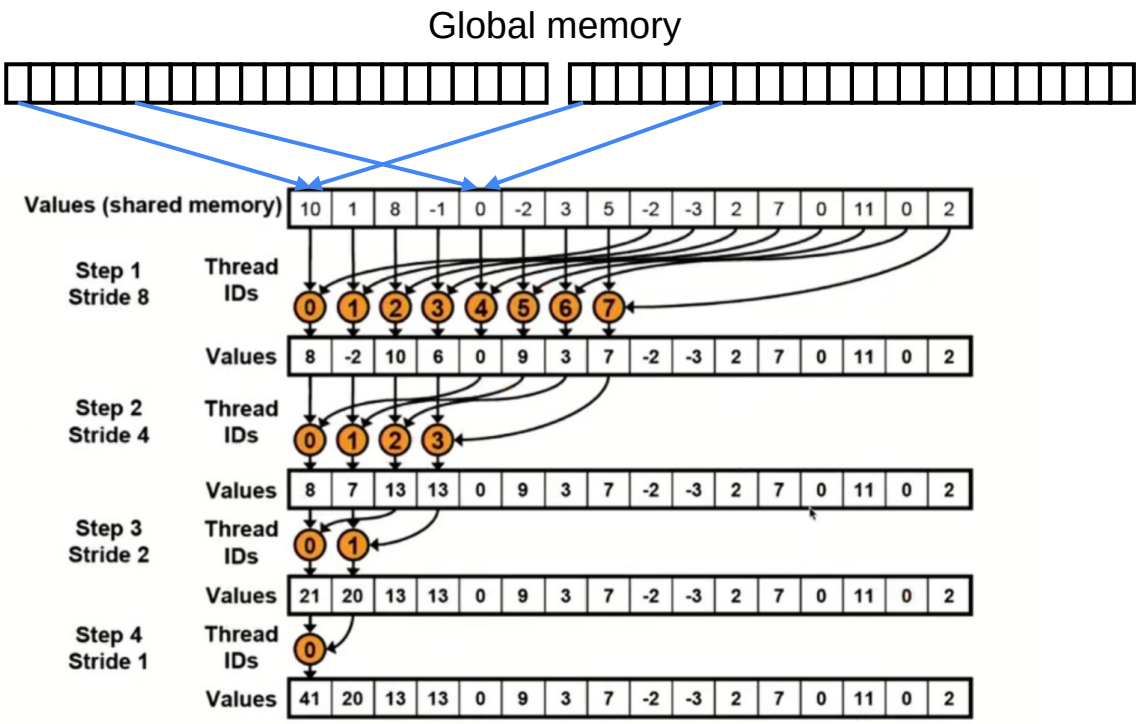


Reduction #3: Sequential Accesses

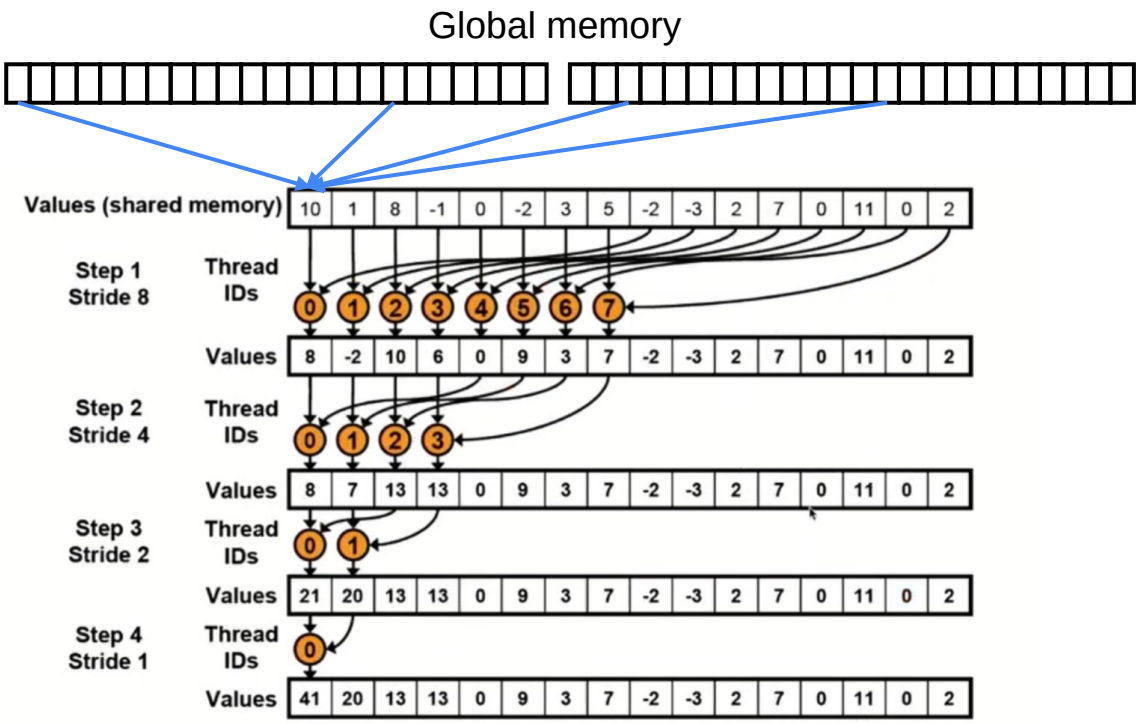


Eliminates bank conflicts

Reduction #4: Load two elements



Reduction #5: Load more elements



Trade-off: Efficiency and SM occupancy

- Loading more elements means higher memory utilization
- However, the number of blocks reduces, and GPU utilization goes down
  - If your number of blocks are lower than the number of SMs, that’s another form of waste
  - You need to profile and find the sweet spot

Useful libraries of high-performance kernels

- cuBLAS: Closed-source GEMM library
- CUTLASS: Open-source template-based GEMM library
- Raft: Vector Search / Clustering / Top-K / Sort
- FlashInfer: Attention (Fused Softmax Discontinuous GEMV)
- CUB: Templates for basic operations at Warp, Block and Device level
  - E.g., reduction

Pybind: use CUDA kernels in python

```
#include <pybind11/pybind11.h>
#include <torch/torch.h>
#include <torch/extension.h>
#include <cuda_runtime.h>

__global__ void add_kernel(int *a, int *b, int *c, size_t num) {
    int block_start = blockIdx.x * blockDim.x;
    int thread_id = threadIdx.x;
    int index = block_start + thread_id;
    if (index < num) {
        c[index] = a[index] + b[index];
    }
}


torch::Tensor add(torch::Tensor a, torch::Tensor b) {
    auto num = a.size(0);
    auto c = torch::empty_like(a);

    int threads_per_block = 256;
    int blocks_per_grid = (num + threads_per_block - 1) / threads_per_block;

    add_kernel<<<blocks_per_grid, threads_per_block>>>(a.data_ptr<int>(), b.data_ptr<int>(), c.data_ptr<int>(), num);
    cudaDeviceSynchronize();
    return c;
}

PYBIND11_MODULE(my_addition, m) {
    m.def("add", &add, "Add two tensors");
}
```